

BROWSERFORGE: SCALING WEB EPISODE VIA PARALLEL BROWSER SANDBOXES

Fei Tang^{1,2}, Huawei Shen², Zhiqiong Lu², Zhengxi Lu¹, Pengyuan Lyu², Chengquan Zhang^{2,†}, Weiming Lu¹, Jun Xiao¹, Yueting Zhuang¹, Yongliang Shen^{1,*},

¹Zhejiang University ²LLM Department, Tencent

*Corresponding author. †Project leader.

ABSTRACT

Web agents that act from rendered pixels avoid the fragility and heavy token cost of reading a page’s HTML or accessibility tree, but training them depends on large amounts of high-quality interaction trajectories, and how to produce such data at scale remains an open problem. Public datasets typically contain only a few thousand trajectories drawn from a fixed and narrow set of websites, and even recent automated synthesis pipelines stay bound to predefined site lists or tutorial sources, so the number of *distinct* websites the agent ever sees barely grows. We present **BrowserForge**, a framework that generates web interaction data at scale by driving many browser sandboxes in parallel over the open web. BrowserForge couples three components: an open-web sourcing stage that exposes the agent to hundreds of thousands of real, openly reachable websites; a sandbox cluster manager that schedules hundreds of concurrent browsers with high utilization; and a Proposer–Solver dual-agent loop that turns a raw page into an executable task and then collects a verified trajectory for it. A rule-plus-model cleaning pipeline removes failed runs and rewrites the surviving reasoning into a single unified chain-of-thought style. Page structure such as the accessibility tree is used only as a synthesis-time signal; the agent we train and release acts purely from the screenshot. The resulting corpus contains 203,238 trajectories, each collected from a distinct website, larger and more diverse than prior trajectory datasets. Fine-tuning a compact multimodal model on this corpus raises its success rate on the live *Online-Mind2Web* from 25.66% to 33.33% and consistently improves step accuracy on the static *Multimodal-Mind2Web*, with the gain growing as the corpus scales. Controlled analyses further confirm that open-web sourcing and broad website coverage are key contributors to the observed improvement.

1 INTRODUCTION

Web agents automate tasks on websites for people: given a goal in natural language, an agent perceives a web page, decides where to act, and carries the goal across many steps of clicking, typing, and navigating until the task is done (Wei et al., 2025; Xiao et al., 2026; Logeswaran et al., 2026; Gur et al., 2024; Deng et al., 2023; Zhou et al., 2023; He et al., 2024a; Wu et al., 2025). As more of daily work moves into the browser, an agent that can operate arbitrary websites on a user’s behalf has become a central goal for multimodal models.

How an agent perceives a page is a design choice. One line of work reads the page as text, namely its HTML or accessibility (a11y) tree, and acts on element identifiers (Zheng et al., 2024a; Deng et al., 2023; Zhou et al., 2023; Lù et al., 2024; Tang et al., 2025); another operates the page from rendered pixels and grounds its actions in screen coordinates, the way a person does (Hong et al., 2024; Wu et al., 2025; Zheng et al., 2024a). The textual route has two liabilities. It is *fragile*: HTML structure and a11y labels differ across sites and shift whenever a page is restyled, so a policy bound to them transfers poorly across the open web (Gou et al., 2025; Lai et al., 2024). And it is *costly*: a single page’s a11y tree routinely serializes to tens of thousands of tokens, inflating context length at every step (Cheng et al., 2024; Gou et al., 2025). A pure-GUI agent avoids both. A screenshot carries the

page at a fixed, far smaller cost, in a form uniform across sites and aligned with how interfaces are built to be used (Gupta et al., 2026; Yang et al., 2026).

Training a capable pure-GUI agent, however, demands interaction trajectories at scale, and crucially over *many distinct websites*, since what the agent learns is bounded by the layouts and interaction patterns it has seen. Public datasets fall short: they contain only a few thousand human demonstrations over tens to low hundreds of sites (Deng et al., 2023; Lù et al., 2024). Automated synthesis pipelines multiply the trajectory *count*, but each stays bound to the source it starts from, whether a tutorial corpus (Xu et al., 2025), a curated seed list (Pahuja et al., 2025), or a fixed environment (Murty et al., 2024; He et al., 2024b), so the agent still sees the same narrow set of sites. The scale that matters for the open web, the number of distinct websites, barely grows.

We build on a simple insight: the open web is itself the largest and cheapest source of *distinct* websites, and a browser sandbox is a cheap, disposable unit of *interaction* with any of them. Run many sandboxes in parallel over openly sourced URLs rather than a fixed list, and scale and diversity grow together, provided the trajectories are verified and cleaned.

On this insight we propose **BrowserForge**, a framework that turns the open web into pure-GUI web-agent training data through three components: a *sourcing and cleaning* stage that draws URLs from Common Crawl (Patel, 2020) and filters them down to hundreds of thousands of real, reachable websites; a *sandbox cluster manager* that schedules hundreds of browsers through a shared work queue at high utilization; and a *Proposer–Solver* loop that turns each page into an executable task and records a trajectory, followed by rule- and model-based cleaning that rewrites the surviving reasoning into a unified chain-of-thought format. Page structure such as the `ally` tree is used only as a synthesis-time signal; the agent we train and release acts purely from the screenshot, so none of it enters inference.

Running this pipeline produces 203,238 trajectories, each collected from a distinct website, larger in count and far broader in coverage than prior trajectory datasets (Table 2). We experiment on two web-agent benchmarks: the live *Online-Mind2Web* (Xue et al., 2025) and the static *Multimodal-Mind2Web*. Fine-tuning a compact backbone on our corpus lifts its Online-Mind2Web success rate from 25.66% to 33.33% (+7.67) and consistently improves Multimodal-Mind2Web step accuracy, reaching a level competitive with much larger open-source web agents. Controlled analyses attribute the gain to the open-web data source and its website diversity rather than to the training recipe, and show performance scaling monotonically with corpus size; ablations confirm that each part of the verification-and-cleaning pipeline contributes. Together these results show that BrowserForge is an effective recipe for producing pure-GUI web-agent data at scale.

We make three contributions:

- We present **BrowserForge**, a data-generation framework that scales web interaction trajectories by orchestrating parallel browser sandboxes over openly sourced URLs, decoupling data scale and diversity from any fixed website list.
- We design a **Proposer–Solver** synthesis loop with an explicit cleaning and verification pipeline that turns unsupervised open-web pages into executable tasks and filtered trajectories with a unified reasoning format.
- We curate a corpus of 203,238 **trajectories, each from a distinct website**, and show that fine-tuning compact multimodal models on it raises Online-Mind2Web success rate to 33.33% and Multimodal-Mind2Web average step accuracy to 43.8%, competitive with much larger open-source web agents.

2 RELATED WORK

Automated trajectory synthesis. Because human annotation of multi-step trajectories is slow and costly, recent work synthesizes trajectories automatically, and this line is the closest to ours. A first group converts existing human-written knowledge into supervision: AgentTrek (Xu et al., 2025) harvests tutorial text from the web, turns it into task specifications, and replays the steps with a vision-language agent while a model verifier checks correctness, and Synatra (Ou et al., 2024) likewise re-purposes indirect knowledge such as online tutorials into direct demonstrations at scale. A second group generates trajectories through interaction: NNetNav (Murty et al., 2024) explores

environments without supervision and relabels the results retroactively; OpenWebVoyager (He et al., 2024b) alternates imitation learning with exploration–feedback cycles; OS-Genesis (Sun et al., 2025) reverses the usual pipeline by acting first and deriving tasks afterward (reverse task synthesis); and Learn-by-interact (Su et al., 2025) synthesizes agent–environment interactions from documentation and reconstructs the matching instructions. Explorer (Pahuja et al., 2025) builds a bottom-up multi-agent pipeline that explores websites dynamically and produces over 94K trajectories, while GUICourse/GUIAct (Chen et al., 2024) contributes large GUI navigation datasets for training VLM agents. These methods substantially increase trajectory counts and report strong downstream gains. Their coverage, however, is still anchored to the source they start from: a tutorial corpus, a curated seed list, or a fixed environment. BrowserForge differs in *where the data comes from*: rather than expanding the number of trajectories within a fixed site distribution, we widen the site distribution itself by sourcing URLs from Common Crawl (Patel, 2020), so that scale and diversity grow together. The Proposer–Solver loop and the cleaning pipeline are what make unsupervised open-web pages usable as training data.

Web agents, benchmarks, and multimodal GUI models. Early LLM-based web agents operated over HTML or accessibility trees, and the shift to multimodal backbones let agents act directly on rendered pages. SeeAct (Zheng et al., 2024a) showed that a strong LMM can plan web actions visually when its plans are grounded onto page elements, and WebVoyager (He et al., 2024a) pushed toward end-to-end agents that interact with live sites. Benchmarks have tracked this shift: Mind2Web (Deng et al., 2023) offers 2,350 crowdsourced tasks over 137 websites; WebLINX (Lù et al., 2024) targets conversational navigation; WebShop (Yao et al., 2022) provides a large simulated shopping environment; WebArena (Zhou et al., 2023) and its multimodal extension VisualWebArena (Koh et al., 2024) host self-contained sites and score functional correctness; GAIA (Mialon et al., 2023) probes general assistants on real-world tool-use and browsing; and Online-Mind2Web (Xue et al., 2025) evaluates agents on live websites and reports that several agents are weaker than earlier numbers suggested. As evaluation suites these are valuable, but as *training* sources they share a ceiling: each draws from a fixed and relatively small set of sites. A parallel line trains end-to-end models that perceive screenshots and emit low-level actions (Tang et al., 2026a; Wu et al., 2025; Tang et al., 2026b; 2025; Gu et al., 2025; Yang et al., 2026). CogAgent (Hong et al., 2024) and SeeClick (Cheng et al., 2024) established screenshot-only GUI grounding and navigation (Tang et al., 2026a; Cheng et al., 2024; Tang et al., 2026c; Lin et al., 2025; Lu et al., 2026a; 2025; 2026b); OS-Atlas (Wu et al., 2025) scales cross-platform grounding data; OmniParser (Lu et al., 2024) parses screenshots into structured elements to ground actions; UGround (Gou et al., 2025) learns universal visual grounding across platforms; Aguvis (Xu et al., 2024) and ShowUI (Lin et al., 2025) build pure-vision GUI agents with unified action spaces; Qwen2.5-VL (Bai et al., 2025) and Qwen3-VL (Qwen Team, 2025) provide strong open backbones; and GUI-specialized models such as UI-TARS (Qin et al., 2025) and UI-TARS-2 (Wang et al., 2025) push success rates on screenshot-driven control. Beyond the web, AndroidControl (Li et al., 2024) and AndroidWorld (Rawles et al., 2024) study data scale and dynamic evaluation for mobile UI control. These models are largely orthogonal to our contribution: BrowserForge produces data, and a stronger backbone or training recipe can be combined with it. In our experiments we fine-tune a compact open multimodal model on the BrowserForge corpus and measure the change in web-agent performance, treating the data pipeline, not the backbone, as the variable under study.

3 METHOD

3.1 OVERVIEW

We introduce **BrowserForge**, a framework that turns the open web into web-agent training data, as shown in Figure 1. The method decomposes into four modules: **(1) Open-web URL sourcing and cleaning** (Section 3.2), which draws candidate URLs from the open web and filters them so that the agent is placed in front of real, openly reachable websites rather than a fixed list; **(2) Parallel browser sandbox orchestration** (Section 3.3), a cluster manager that drives hundreds of browser sandboxes at once and supplies rendered pages at high throughput; **(3) Proposer–Solver task synthesis** (Section 3.4), in which a Proposer agent turns a rendered page into an executable task and a Solver agent carries it out and records the trajectory; and **(4) Trajectory cleaning and unified reasoning** (Section 3.5), which filters the collected trajectories and rewrites their reasoning into a

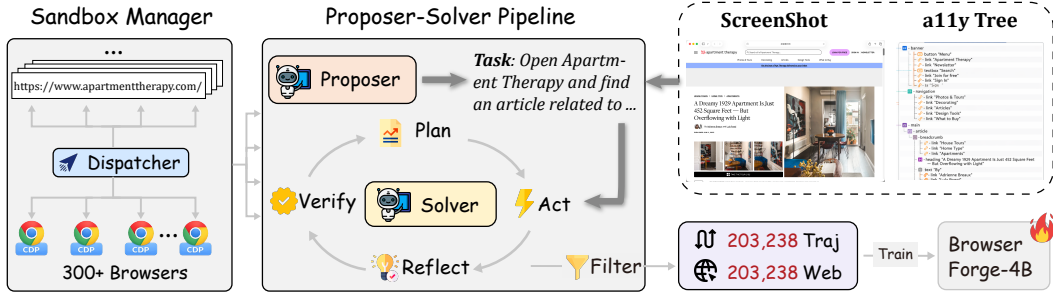


Figure 1: The BrowserForge synthesis framework. A *sandbox cluster manager* (left) maintains a machine list, a URL work queue, a pool of browser sandboxes, and worker threads. For each page, the *Proposer* (middle) generates a candidate task in two stages and selects the most executable one; the *Solver* (right), built on an open browser-control agent, executes the task through a plan–act–reflect–verify loop and writes the trajectory to storage. Verified trajectories are cleaned and formatted into the final training corpus.

single format. Together these modules target three properties at once: *scale*, from parallel sandboxes; *diversity*, from open-web URL sourcing; and *quality*, from verification and cleaning.

3.2 OPEN-WEB URL SOURCING AND CLEANING

Prior trajectory-synthesis methods focus on generating more trajectories, but they remain bound to the source they start from, whether a curated seed list, a tutorial corpus, or a fixed environment, which caps the number of distinct websites the agent ever sees. We instead build the corpus on *real* URLs. Our sourcing is inspired by Common Crawl (Patel, 2020), a large-scale, periodically refreshed snapshot of the public web that spans many millions of hosts; sampling from such a snapshot rather than a fixed list means the supply of candidate URLs can, in principle, be scaled up without bound, and it is what lets BrowserForge reach hundreds of thousands of distinct websites.

Raw Common Crawl URLs, however, are uncleaned: many are dead, toxic, or static endpoints that afford no interaction. We therefore design a cleaning pipeline that runs before any sandbox time is spent, combining standard filters with a reachability check tailored to our own infrastructure:

- **Reachability and type filtering.** We discard URLs that fail to resolve or load, and filter by site type to keep pages that afford interaction rather than static endpoints (for example, raw media or file downloads).
- **Content filtering.** Very short or near-empty pages carry little interaction signal and are removed; we keep pages whose main content and multimodal elements support a non-trivial task.
- **Blacklist filtering.** We remove URLs that overlap with evaluation benchmarks or known sensitive hosts, which keeps the training distribution clean and avoids contamination of the test sets used in Section 4.
- **IP-level accessibility validation.** A URL that is reachable in general may still be blocked from, or fail to render on, the machines that run our sandboxes. We therefore re-validate each surviving URL from the cluster’s own IP addresses and keep only those that remain actually accessible, so that no sandbox time is wasted on pages it cannot open.

Each surviving URL is rendered inside a sandbox, and the resulting page state is packaged for the synthesis stage as a tuple of the current screenshot, the page URL and title, and the accessibility (a11y) tree. To broaden the visual and locale distribution further, sandboxes are configured with varied screen resolutions, user agents, and interface languages, which diversifies image sizes and rendered layouts without changing the underlying task logic.

3.3 PARALLEL BROWSER SANDBOX ORCHESTRATION

Scale comes from running many browser sandboxes concurrently, and doing so without leaving compute idle requires explicit coordination. A *sandbox cluster manager* owns this fleet and keeps

Action	Arguments	Description
click	coordinate	Click a point on the screen.
type	coordinate, text	Focus an input field and type text.
select	coordinate, value	Choose a value from a dropdown / combobox.
scroll	direction, amount	Scroll the view up/down/left/right by a fraction of the viewport.
key	key	Press a single key (e.g. Enter, Tab, Escape).
wait	seconds	Wait for the page to load or settle.
go_back	—	Navigate back to the previous page.
visit_url	url	Navigate the current tab to an absolute URL.
finish	message	Conclude the task and return the answer.

Table 1: The Solver’s unified action space. Screen positions are expressed as normalized integer coordinates in $[0, 1000]$, independent of the rendered resolution. At each step the Solver emits exactly one action from this set; the `finish` action terminates the trajectory.

it busy. It maintains four pieces of state: a machine list of available compute nodes; a shared URL work queue holding the cleaned pages waiting to be processed; a sandbox pool of live browsers, each isolated from the others and addressable through the Chrome DevTools Protocol (CDP); and a worker pool of threads, each of which repeatedly claims a free sandbox, pulls the next URL from the queue, drives the synthesis loop on that page, writes the resulting trajectory back to storage, and then returns the sandbox to the pool for reuse.

This design scales to up to 300 browser sandboxes in parallel. The shared queue is the key to high utilization: because workers pull work rather than being assigned fixed partitions, a slow or stalled page occupies only its own worker, and the rest of the fleet keeps draining the queue instead of waiting on it. Each sandbox can host several independent browser profiles, so a single node runs multiple concurrent sessions and the cost of browser startup is amortized across many pages. Compute nodes can be added to the machine list while the pipeline is running, and the queue rebalances onto them automatically. This orchestration layer is what turns a single-machine data collector into a cluster-scale one, and it is the component that most directly determines throughput.

3.4 PROPOSER–SOLVER TASK SYNTHESIS

Open-web sourcing supplies pages, but a rendered page is not yet training data: it has no task to pursue and no trajectory to learn from. What is missing is a *task source*. We supply one with a **Proposer–Solver** framework: the Proposer turns a page into an executable task, and the Solver carries that task out and records the trajectory.

Proposer. The Proposer reads the packaged page state (the accessibility tree, the screenshot, and the page URL and title) and produces an executable task in two stages. In the *proposal* stage, we prompt Qwen3-VL-235B (Qwen Team, 2025) to generate three candidate tasks that are grounded in the visible page elements and that could plausibly be completed on this page; the prompt explicitly excludes tasks whose keywords require account registration, login, or payment, since such tasks cannot be completed autonomously and are unsafe to attempt. In the *reflection-and-selection* stage, the model re-examines the three candidates against the page, judges which are genuinely executable, and selects a single best task. Proposing several candidates and then selecting among them, rather than committing to the first idea, filters out under-specified or impossible tasks before any execution effort is spent and raises the yield of usable trajectories.

Solver. Given the selected task, the Solver collects a trajectory by acting on the live page. It is built on an open browser-control agent (Müller & Žunič, 2024) and driven by a multimodal model through a closed loop of *planning*, *acting*, *reflection*, and *verification*, with a running *memory* of prior steps. At each step the model observes the current page as a screenshot together with the structured set of interactive elements, and emits an evaluation of the previous step, an updated memory, the next goal, and one or more grounded actions (such as clicking an element, typing text, or scrolling), which are executed in the sandbox. The Solver acts through a unified action space, summarized in Table 1, in which screen positions are expressed as normalized integer coordinates so that the same action format

applies across pages of different resolutions. The verification component checks whether the task has been satisfied and decides whether to continue or to stop; a dedicated terminal action signals completion and ends the run. Because observation, reasoning, and action are interleaved within a single agent loop, the Solver can recover from intermediate mistakes rather than committing to a fixed plan.

From runs to a corpus. Each completed run is stored as a structured trajectory, the agent’s running memory together with a per-step JSON record of observations, thoughts, and actions, and accumulated into the training set. Because the Proposer and Solver both run inside the sandbox fleet, task generation and trajectory collection scale with the same parallel infrastructure described in Section 3.3, so the pipeline produces a large corpus of raw trajectories at cluster throughput. In total we collect 203,238 raw trajectories, one per distinct website, averaging 8.8 steps each, for roughly 1.8M interaction steps before any filtering.

3.5 TRAJECTORY CLEANING AND UNIFIED REASONING FORMAT

The pipeline so far yields trajectories at scale, but because tasks are proposed and executed without human supervision, a fraction of the sampled runs fail: the Solver may pursue a goal that turns out to be impossible, take a broken action sequence, or stop prematurely, and the recorded reasoning varies widely in style and length from run to run. Training directly on this raw mixture passes the noise into the model. We therefore clean the corpus in three steps: the first two filter the raw collection for correctness, and the third standardizes the reasoning of the data we ultimately train on.

- **Rule-based filtering.** We check whether the trajectory’s final action is the terminal `Finish` action. Runs that never terminate properly, or whose action sequences are malformed, are discarded outright, removing obvious failures cheaply before any model is invoked.
- **Model-based judging.** For the surviving runs, we prompt Qwen3-VL-235B with the task description together with the last three screenshots of the trajectory, and ask it to judge whether the task was actually accomplished. Trajectories the judge rejects are removed, which filters out runs that terminate cleanly but do not in fact complete the task. Together these two filters are strict: only about 30% of the raw interaction steps survive, leaving roughly 600K verified steps.
- **Unified chain-of-thought rewriting.** From this verified pool we sample 200K steps and use Seed 2.0 Pro to rewrite their reasoning into a single, consistent chain-of-thought format. The Solver’s original rationales are often overly verbose and heterogeneous; rewriting them gives the downstream model a stable supervision target rather than a mix of inconsistent reasoning styles. These 200K Seed-rewritten steps form the final training corpus.

Overall, BrowserForge sources 203,238 raw trajectories from as many distinct websites; filtering keeps roughly 30% of their steps, and from this verified pool we draw the 200K Seed-rewritten steps used for training. Table 2 places this corpus against prior web-agent trajectory datasets along scale, site coverage, and the source of diversity.

4 EXPERIMENTS

Training a compact multimodal model on the BrowserForge corpus turns it into a markedly stronger web agent, and controlled experiments trace that gain to the corpus itself rather than to the training recipe. We organize the evaluation around this claim: a main comparison on a live benchmark (Section 4.2), an analysis that isolates the data source and scale (Section 4.3), an ablation of the synthesis and cleaning pipeline (Section 4.4), and an error analysis of how the trained agent fails (Section 4.5).

4.1 EXPERIMENTAL SETUP

Implementation details. We fine-tune the compact open Qwen3.5 multimodal models at 4B and 9B to obtain **BrowserForge-4B** and **BrowserForge-9B**. Training runs on the LLaMA-Factory framework (Zheng et al., 2024b) for 3 epochs at a learning rate of $1e-5$, with a maximum of

Dataset	Scale		Diversity	Reason
	#Trajectories	#Websites		
Mind2Web (Deng et al., 2023)	1,009	137	Fixed	×
WebLINX (Lù et al., 2024)	969	155	Fixed	×
GUIAct (Chen et al., 2024)	2,482	121	Fixed	×
OpenWebVoyager (He et al., 2024b)	1,165	48	Fixed	✓
NNetNav (Murty et al., 2024)	10,272	20	Fixed	✓
AgentTrek (Xu et al., 2025)	10,398	127	Tutorial	✓
Explorer (Pahuja et al., 2025)	94,000	49,000	Explore	✓
BrowserForge (Ours)	203,238	~200K	Common Crawl	✓

Table 2: Comparison of BrowserForge with existing web-agent trajectory datasets. “Reason” marks whether task synthesis includes an explicit reasoning/verification step. Each BrowserForge trajectory is collected from a distinct website, so its trajectory and website counts coincide; it is the largest in trajectory count and the broadest in website coverage, sourcing sites from the open web rather than a fixed list.

99,999,999 pixels per image so that pages are effectively left uncropped. We freeze the visual encoder and the adapter (projection) layers and perform full-parameter supervised fine-tuning of the language-model component only, supervising the assistant response tokens. Training and inference share one fixed system prompt that conditions on the current screenshot and emits a single action from the unified action space per step (Appendix A.1).

Benchmarks and metric. We evaluate on two complementary benchmarks. *Online-Mind2Web* (Xue et al., 2025) measures end-to-end task success on 300 tasks over live websites; following prior practice, success is scored by the WebJudge-7B (Xue et al., 2025) automatic judge and we report success rate (SR, %). *Multimodal-Mind2Web* scores *step-wise* success across its Cross-Task, Cross-Website, and Cross-Domain splits under a fixed-trajectory protocol, probing fine-grained grounding rather than full task completion.

Baselines. We compare against two groups: (i) proprietary GUI agents, shown only as an upper-reference point rather than as controlled comparisons; and (ii) open multimodal models, including Qwen3-VL and the Qwen3.5 series evaluated zero-shot, alongside open-source GUI and web-agent methods.

4.2 MAIN RESULTS

We evaluate on two mainstream web-agent benchmarks, *Online-Mind2Web*, a dynamic benchmark that measures end-to-end task success on live websites, and *Multimodal-Mind2Web*, a static benchmark that measures step-wise accuracy under a fixed-trajectory protocol. As shown in Table 3 and Table 4, the results support three findings:

(1) BrowserForge data yields a consistent gain on both the dynamic and the static benchmark. On the dynamic *Online-Mind2Web*, fine-tuning Qwen3.5-4B on the BrowserForge corpus raises success rate from 25.66% to 33.33% (+7.67), and Qwen3.5-9B from 29.33% to 38.00% (+9.33). On the static *Multimodal-Mind2Web*, average step accuracy rises in lockstep, from 38.2% to 43.8% (Pass@1) for the 4B model and from 40.0% to 45.1% for the 9B model. The improvement appears on both protocols and both backbones rather than at a single setting.

(2) A compact BrowserForge agent surpasses much larger open-source models and competitive proprietary ones. On *Online-Mind2Web*, BrowserForge-4B already overtakes larger open-source agents such as ScaleCUA-7B (30.3%) and GUI-Libra-4B (31.3%), and BrowserForge-9B at 38.0% exceeds the strongest prior open-source method GUI-Libra-8B (36.7%). Notably, GUI-Libra is trained with reinforcement learning, whereas BrowserForge uses supervised fine-tuning only, so the gain comes from the data rather than a heavier training recipe. The 9B model also clears strong proprietary references, including GPT-5 with UGround (33.3%) and Browser-Use (30.0%). That a 4B/9B model reaches this level points to data quality, not parameter count, as the driver.

(3) The gain is robust across distribution shifts. On *Multimodal-Mind2Web* (Table 4), the improvement holds on every split, Cross-Task, Cross-Website, and Cross-Domain, rather than coming

Model	Domain	Agent Traj.	SR (%)
<i>Proprietary Models</i>			
Browser-Use	Web Agent	N/A	30.0
Gemini-CUA	Computer-Use Agent	N/A	69.0
Navigator	Web Agent	N/A	78.7
Operator	Computer-Use Agent	Undisc.	61.3
GPT-5 + UGround-v1-7B	Planner & Grounder	N/A	33.3
<i>Base MLLMs</i>			
Qwen2.5-VL-3B	General VLM	N/A	8.3
Qwen2.5-VL-7B	General VLM	N/A	22.0
Qwen2.5-VL-32B	General VLM	N/A	19.7
Qwen3-VL-4B	General VLM	N/A	27.7
Qwen3-VL-8B	General VLM	N/A	27.7
Qwen3-VL-32B	General VLM	N/A	34.3
<i>Existing Methods</i>			
ScaleCUA-3B	Computer-Use Agent	19K	21.7
ScaleCUA-7B	Computer-Use Agent	19K	30.3
ScaleCUA-32B	Computer-Use Agent	19K	30.0
GUI-Libra-3B	Web Agent	9K	29.0
GUI-Libra-4B	Web Agent	9K	31.3
GUI-Libra-7B	Web Agent	9K	33.3
GUI-Libra-8B	Web Agent	9K	36.7
WebStar-7B	Web Agent	13.3K	22.8
WebStar-32B	Web Agent	13.3K	23.8
<i>Ours</i>			
Qwen3.5-4B (baseline)	General VLM	N/A	25.7
BrowserForge-4B	Web Agent	20K	33.3 ($\uparrow 7.6$)
Qwen3.5-9B (baseline)	General VLM	N/A	29.3
BrowserForge-9B	Web Agent	20K	38.0 ($\uparrow 9.3$)

Table 3: Success rate (SR, %) on Online-Mind2Web, scored using the WebJudge-7B automatic judge. *Agent Traj.* denotes the reported number of agent-interaction trajectories used for specialized training. “Undisc.” indicates that the training scale is not publicly disclosed, while “–” indicates that no specialized agent trajectories are used. Proprietary models are included for reference only. **Bold** denotes the best result among the compared base models, existing methods, and our models; underlining denotes the best prior result. BrowserForge-4B/9B are fine-tuned from Qwen3.5-4B/9B on the BrowserForge corpus, with absolute gains over the corresponding backbones shown in green.

from one favorable distribution. BrowserForge-4B improves Pass@4 average step accuracy over its backbone from 44.1% to 54.4% and BrowserForge-9B from 47.2% to 56.3%, with the scaling trend shown in Figure 2(a).

In summary, training on BrowserForge data with supervised fine-tuning alone turns a compact backbone into a web agent that improves consistently across dynamic and static benchmarks and across distribution shifts, reaching a level competitive with far larger open-source and proprietary systems.

4.3 ANALYSIS

The main results establish that BrowserForge data helps; the central claim of this paper, however, is about *where* the data comes from. We therefore run two controlled experiments that isolate the data source and its scale from the model and optimizer.

Source: the open-web corpus, not the training recipe, drives the gain. A natural concern is that any large fine-tuning set would help equally well. To rule this out, we hold the Qwen3.5-4B backbone and the 3-epoch budget fixed and vary only the data, fine-tuning on existing open-source web trajectories versus the BrowserForge corpus. For a fair comparison, we sample 200K trajectories from the open-source data to match the size of our training set. As shown in Table 5, training on BrowserForge data raises average step accuracy on Multimodal-Mind2Web from 32.74% to 41.33%

Model	Cross-Task		Cross-Website		Cross-Domain		Average	
	Pass@1	Pass@4	Pass@1	Pass@4	Pass@1	Pass@4	Pass@1	Pass@4
<i>Proprietary Models with SeeAct-V Framework</i>								
GPT-4o + UGround-v1-7B	35.7	38.9	33.9	37.6	39.1	42.2	36.2	39.6
GPT-4.1 + UGround-v1-7B	41.1	44.8	36.2	39.7	43.0	46.4	40.1	43.6
GPT-5-mini + UGround-v1-7B	44.2	48.0	40.4	44.2	45.8	48.1	43.5	46.7
GPT-5 + UGround-v1-7B	47.7	51.7	45.0	47.6	48.2	51.6	47.0	50.3
<i>Open-source Native Models</i>								
GUI-R1-3B	24.0	37.7	22.3	37.7	24.6	39.1	23.6	38.2
GUI-R1-7B	37.0	50.1	34.1	46.3	39.6	50.5	36.9	49.0
Aguvis-7B	37.7	48.0	31.7	41.5	36.9	45.1	35.4	44.9
UI-TARS-1.5-7B	37.2	48.0	31.3	42.9	35.6	47.2	34.7	46.0
GLM-4.1V-9B-Thinking	26.9	32.9	23.0	29.3	28.7	35.3	26.2	32.5
Qwen2.5-VL-3B	24.4	29.0	18.6	24.6	27.1	31.2	23.4	28.3
Qwen2.5-VL-7B	31.6	45.6	30.4	42.1	35.6	48.0	32.5	45.2
Qwen2.5-VL-32B	46.2	55.8	42.6	55.7	46.0	57.9	44.9	56.5
Qwen3-VL-4B	43.5	49.9	39.1	48.7	43.2	50.1	41.9	49.6
Qwen3-VL-8B	45.9	52.8	40.0	48.3	41.8	52.3	42.6	51.1
Qwen3.5-4B	39.5	45.1	35.1	40.8	40.0	46.3	38.2	44.1
Qwen3.5-9B	42.1	48.3	36.5	43.6	41.5	49.7	40.0	47.2
BrowserForge-4B (Ours)	47.0	56.8	41.5	52.4	42.9	54.0	43.8	54.4
BrowserForge-9B (Ours)	48.1	57.9	41.3	54.6	45.9	56.5	45.1	56.3

Table 4: Step accuracy (%) on Multimodal-Mind2Web. Proprietary models are shown for reference. **Bold:** best; underline: second best among open-source methods. BrowserForge-4B/9B are fine-tuned from Qwen3.5-4B/9B on the BrowserForge corpus and improve over their respective backbones on every split.

Training data	Cross-Task		Cross-Website		Cross-Domain		Average	
	Pass@1	Pass@4	Pass@1	Pass@4	Pass@1	Pass@4	Pass@1	Pass@4
Open-source trajectories	36.45	45.41	28.85	41.02	32.93	44.01	32.74	43.48
BrowserForge (Ours)	44.88	54.74	37.88	50.83	41.22	52.20	41.33	52.59

Table 5: Controlled comparison of the training-data source on Multimodal-Mind2Web step accuracy (%). Holding the Qwen3.5-4B backbone and the 3-epoch training budget fixed, we fine-tune on existing open-source web trajectories versus the BrowserForge corpus. Read across the two rows: the gap isolates the effect of the data source rather than the architecture or optimizer.

(Pass@1) and from 43.48% to 52.59% (Pass@4), a gain of close to nine percent that holds across all three splits. Because everything but the data is identical, the gain is attributable to the corpus, to its open-web diversity and the rule-plus-model cleaning, rather than to the architecture or optimizer.

Scale: performance grows monotonically with the amount of BrowserForge data. If open-web sourcing is the right lever, more of it should yield a better agent. We fine-tune the same backbone on nested subsets of the $\sim 200\text{K}$ unified training set (10^3 , 5×10^4 , 10^5 , and the full 2×10^5 samples) under an identical 3-epoch budget, against the zero-shot base model (0 samples). Figure 2(a) plots the trend on both benchmarks. Performance rises monotonically throughout: Online-Mind2Web success climbs from 25.66% at zero-shot to 27.33% at 10^5 and 33.33% at the full set, while Multimodal-Mind2Web step accuracy rises from 23.11% to 41.83% and 54.36% over the same range. The curve follows an approximately power-law shape with no sign of saturating at 2×10^5 , supporting the premise that open-web sourcing keeps improving the agent as the corpus grows.

4.4 ABLATION

The cleaning pipeline is what turns the raw, noisy trajectories into verified supervision with a consistent reasoning format, so we ask how much each of its stages actually contributes to the trained

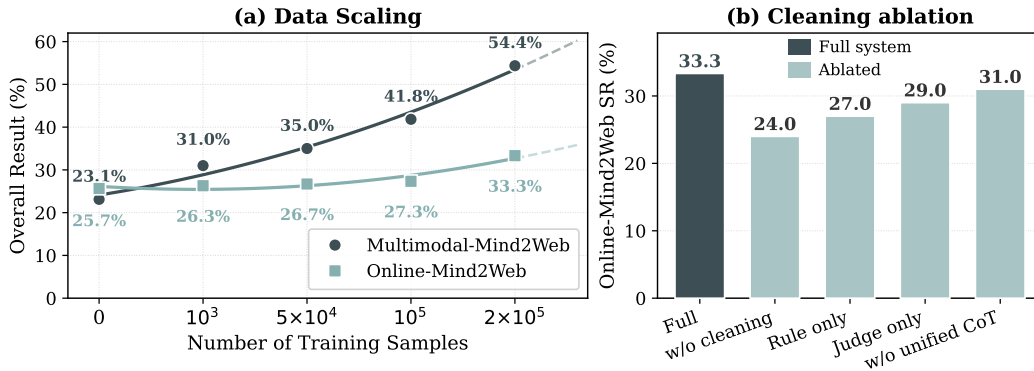


Figure 2: (a) Data scaling. Overall result versus the number of BrowserForge training samples (log scale), fine-tuning Qwen3.5-4B for 3 epochs under a fixed budget, on both Multimodal-Mind2Web (step accuracy) and Online-Mind2Web (success rate). The 0-sample point is the zero-shot base model; solid curves are power-law fits over the measured points and dashed segments extrapolate beyond the largest run. **(b) Cleaning ablation.** Online-Mind2Web success rate for each variant of the cleaning pipeline (rule-based filter, model-based judge, and Seed-rewritten unified chain-of-thought), with the full system highlighted.

agent. Starting from the full system, which applies the rule-based filter, the model-based judge, and the Seed-rewritten unified chain-of-thought in sequence, we construct four reduced variants that each train the same Qwen3.5-4B for the same 3 epochs and differ only in how the data is cleaned: (i) *w/o cleaning*, trained on the raw trajectories with no filtering or rewriting; (ii) *rule-based filter only*, which keeps trajectories that terminate with a valid `Finish` action but applies neither the judge nor the rewriting; (iii) *model judge only*, which keeps trajectories the judge accepts but skips the rule filter and the rewriting; and (iv) *w/o unified CoT*, which applies both filters but trains on the Solver’s original reasoning instead of the unified format. All variants are evaluated by success rate on Online-Mind2Web under the identical protocol used above.

As shown in Figure 2(b), we draw three findings: **(1) Verification is essential.** Removing cleaning entirely and training on raw trajectories gives the weakest agent (24.0%), well below the full system (33.3%); the noise from failed and incomplete runs passes directly into the model. **(2) The two filters are complementary.** Applying only the rule filter (27.0%) or only the model judge (29.0%) each recovers part of the gap but neither matches using both, since the rule filter removes malformed runs cheaply while the judge catches runs that terminate cleanly yet do not complete the task. **(3) A unified reasoning format adds a further gain.** Retaining both filters but dropping the Seed-rewritten chain-of-thought (31.0%) trails the full system, showing that a consistent supervision target, not just correct trajectories, helps the model learn. Overall, every stage of the pipeline contributes, and the full combination of filtering and unified-CoT rewriting yields the strongest agent at 33.3%.

4.5 ERROR ANALYSIS

Most failures are not random: the agent gets stuck in unproductive repetition, pointing to a missing feedback signal rather than a missing skill. We sample 100 Online-Mind2Web trajectories of BrowserForge-4B and bucket each into a single outcome with a priority rule. Only 11% terminate cleanly; the rest fall into a few recurring patterns, as shown in Table 6. (1) **Click loop** (29%) is a perception gap: the agent clicks the same coordinate four or more times while the page does not change, mis-targeting an element and failing to notice the action had no effect, so it does not ground the consequence of an action in the next observation and repeats the identical click instead of switching strategies. (2) **Aimless back-navigation** (22% and 20%) shows the same pattern in navigation: `GoBack` signals self-monitoring, but looped it degrades into ineffective exploration, with 22% of runs issuing 3+ `GoBack` actions and 20% ending in 5+ consecutive `Scroll` actions, oscillating between the two without committing to a concrete sub-goal. (3) **Access blocking** (9%) is an environment limit: the reasoning explicitly mentions an access wall (“access denied”, a Cloudflare interstitial, or a CAPTCHA), which the judge counts as a failure even though the cause is external and the agent correctly perceives the block, so the gap is the inability to act on it rather than to recognize it. (4) **Step-limit timeout** (8%) occurs when a run exhausts the 30-step budget while still making

Outcome	Description	%
Task completed	Terminates with <code>Finish/Exit</code> or a final answer	11.0
Click loop	Same coordinate clicked ≥ 4 times with no state change	29.0
Back loop	≥ 3 <code>GoBack</code> actions (often paired with scrolling)	22.0
Scroll-only tail	Run ends with ≥ 5 consecutive <code>Scroll</code> actions	20.0
Access blocked	Reasoning mentions access denied / CAPTCHA / bot wall	9.0
Step limit	Exhausts the 30-step budget without an above pattern	8.0
No action	Empty trajectory (no action emitted)	1.0

Table 6: Outcome distribution over 100 BrowserForge-4B trajectories sampled from Online-Mind2Web. The top row is the only success category; the remaining rows are mutually exclusive failure modes, assigned by a priority rule and ordered by frequency. The three dominant failures, namely repeated clicking, repeated back-navigation, and trailing scrolls, are all forms of *unproductive repetition* that together account for 71% of trajectories.

plausible progress, so the cap is a partly artificial source of failure that makes the reported success rate a conservative lower bound.

5 CONCLUSION

We presented BrowserForge, a framework that generates web episodes at scale by driving hundreds of browser sandboxes in parallel over openly sourced URLs. By drawing candidate sites from the open web, scheduling sandboxes through a shared work queue, and pairing a Proposer that writes executable tasks with a Solver that collects verified trajectories, BrowserForge decouples the scale and diversity of web-agent data from any fixed website list. A rule-plus-model cleaning pipeline and a unified reasoning format turn the raw, noisy trajectories into verified training data with a consistent supervision target. The resulting corpus of 203,238 trajectories, each from a distinct website, is larger and broader than prior trajectory datasets. Fine-tuning a compact multimodal model on it raises success rate on the live Online-Mind2Web benchmark from 25.66% to 33.33% and improves step accuracy on the static Multimodal-Mind2Web benchmark, and controlled analyses trace the gain to the open-web data source and its website diversity rather than to the training recipe alone.

REFERENCES

- Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2.5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025.
- Wentong Chen, Junbo Cui, Jinyi Hu, Yujia Qin, et al. Guicourse: From general vision language models to versatile gui agents. *arXiv preprint arXiv:2406.11317*, 2024.
- Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing gui grounding for advanced visual gui agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.
- Boyuan Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for GUI agents. In *International Conference on Learning Representations (ICLR)*, 2025.
- Zhangxuan Gu, Zhengwen Zeng, Zhenyu Xu, Xingran Zhou, Shuheng Shen, Yunfei Liu, Beitong Zhou, Changhua Meng, Tianyu Xia, Weizhi Chen, et al. Ui-venus technical report: Building high-performance ui agents with rft. *arXiv preprint arXiv:2508.10833*, 2025.
- Tanmay Gupta, Piper Wolters, Zixian Ma, Peter Sushko, Rock Yuren Pang, Diego Llanes, Yue Yang, Taira Anderson, Boyuan Zheng, Zhongzheng Ren, et al. Molmoweb: Open visual web agent and open data for the open web. *arXiv preprint arXiv:2604.08516*, 2026.

-
- Izzeddin Gur, Hiroki Furuta, Austin Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. A real-world webagent with planning, long context understanding, and program synthesis. In *International Conference on Learning Representations*, volume 2024, pp. 52690–52717, 2024.
- Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024a.
- Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Hongming Zhang, Tianqing Fang, Zhenzhong Lan, and Dong Yu. Openwebvoyager: Building multimodal web agents via iterative real-world exploration, feedback and optimization. *arXiv preprint arXiv:2410.19609*, 2024b.
- Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, and Jie Tang. Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- Hanyu Lai, Xiao Liu, Iat Long Iong, Shuntian Yao, Yuxuan Chen, Pengbo Shen, Hao Yu, Hanchen Zhang, Xiaohan Zhang, Yuxiao Dong, et al. Autowebglm: A large language model-based web navigating agent. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 5295–5306, 2024.
- Wei Li, William E. Bishop, Alice Li, Christopher Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. On the effects of data scale on ui control agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou. Showui: One vision-language-action model for gui visual agent. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- Lajanugen Logeswaran, Jaekyeom Kim, Sungryull Sohn, Creighton Glasscock, and Honglak Lee. Scaling web agent training through automatic data generation and fine-grained evaluation. *arXiv preprint arXiv:2602.12544*, 2026.
- Xing Han Lù, Zdeněk Kasner, and Siva Reddy. Weblinx: Real-world website navigation with multi-turn dialogue. In *International Conference on Machine Learning (ICML)*, 2024.
- Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. Omniparser for pure vision based gui agent. *arXiv preprint arXiv:2408.00203*, 2024.
- Zhengxi Lu, Jiabo Ye, Fei Tang, Yongliang Shen, Haiyang Xu, Ziwei Zheng, Weiming Lu, Ming Yan, Fei Huang, Jun Xiao, et al. Ui-s1: Advancing gui automation via semi-online reinforcement learning. *arXiv preprint arXiv:2509.11543*, 2025.
- Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Pengxiang Zhao, Guangyi Liu, et al. Ui-r1: Enhancing efficient action prediction of gui agents by reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 17608–17616, 2026a.
- Zhengxi Lu, Fei Tang, Guangyi Liu, Jin Ma, Kaitao Song, Xu Tan, Wenqi Zhang, Weiming Lu, Jun Xiao, Yueting Zhuang, et al. Ui-copilot: Advancing long-horizon gui automation via tool-integrated policy optimization. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 19741–19762, 2026b.
- Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: a benchmark for general ai assistants. *arXiv preprint arXiv:2311.12983*, 2023.

-
- Magnus Müller and Gregor Žunič. Browser use: Enable ai to control your browser. <https://github.com/browser-use/browser-use>, 2024.
- Shikhar Murty, Hao Zhu, Dzmitry Bahdanau, and Christopher D. Manning. Nnetnav: Unsupervised learning of browser agents through environment interaction in the wild. *arXiv preprint arXiv:2410.02907*, 2024.
- Tianyue Ou, Frank F. Xu, Aman Madaan, Jiarui Liu, Robert Lo, Abishek Sridhar, Sudipta Sengupta, Dan Roth, Graham Neubig, and Shuyan Zhou. Synatra: Turning indirect knowledge into direct demonstrations for digital agents at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2409.15637.
- Vardaan Pahuja, Yadong Lu, Corby Rosset, Boyu Gou, Arindam Mitra, Spencer Whitehead, Yu Su, and Ahmed Hassan Awadallah. Explorer: Scaling exploration-driven web trajectory synthesis for multimodal web agents. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 6300–6323, 2025.
- Jay M Patel. Introduction to common crawl datasets. In *Getting structured data from the internet: running web crawlers/scrapers on a big data production scale*, pp. 277–324. Springer, 2020.
- Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.
- Qwen Team. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. Androidworld: A dynamic benchmarking environment for autonomous agents. *arXiv preprint arXiv:2405.14573*, 2024.
- Hongjin Su, Ruoxi Sun, Jinsung Yoon, Pengcheng Yin, Tao Yu, and Sercan Ö. Arik. Learn-by-interact: A data-centric framework for self-adaptive agents in realistic environments. In *International Conference on Learning Representations (ICLR)*, 2025.
- Qiushi Sun, Kanzhi Cheng, Zichen Ding, Chuanyang Jin, Yian Wang, Fangzhi Xu, Zhenyu Wu, Chengyou Jia, Liheng Chen, Zhoumianze Liu, et al. Os-genesis: Automating gui agent trajectory construction via reverse task synthesis. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2412.19723.
- Fei Tang, Haolei Xu, Hang Zhang, Siqi Chen, Xingyu Wu, Yongliang Shen, Wenqi Zhang, Guiyang Hou, Zeqi Tan, Yuchen Yan, et al. A survey on (m) llm-based gui agents. *arXiv preprint arXiv:2504.13865*, 2025.
- Fei Tang, Zhangxuan Gu, Zhengxi Lu, Xuyang Liu, Shuheng Shen, Changhua Meng, Wen Wang, Wenqi Zhang, Yongliang Shen, Weiming Lu, et al. Gui-g²: Gaussian reward modeling for gui grounding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 33214–33222, 2026a.
- Fei Tang, Zhangxuan Gu, Zhengxi Lu, Shangzhan Zhang, Zhengwen Zeng, Shuheng Shen, Changhua Meng, Yuchen Yan, Wenqi Zhang, Yongliang Shen, et al. Gui-sage: Enhancing gui automation with self-explanatory learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 13007–13016, 2026b.
- Fei Tang, Zhiqiong Lu, Boxuan Zhang, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. Clawgui: A unified framework for training, evaluating, and deploying gui agents. *arXiv preprint arXiv:2604.11784*, 2026c.
- Haoming Wang et al. Ui-tars-2 technical report: Advancing gui agent with multi-turn reinforcement learning. *arXiv preprint arXiv:2509.02544*, 2025.

-
- Zhepei Wei, Wenlin Yao, Yao Liu, Weizhi Zhang, Qin Lu, Liang Qiu, Changlong Yu, Puyang Xu, Chao Zhang, Bing Yin, et al. Webagent-r1: Training web agents via end-to-end multi-turn reinforcement learning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 7920–7939, 2025.
- Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. Os-atlas: A foundation action model for generalist gui agents. In *International Conference on Learning Representations (ICLR)*, 2025.
- Zikai Xiao, Jianhong Tu, Chuhan Zou, Yuxin Zuo, Zhi Li, Peng Wang, Bowen Yu, Fei Huang, Junyang Lin, and Zuozhu Liu. Webworld: A large-scale world model for web agent training. *arXiv preprint arXiv:2602.14721*, 2026.
- Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong. Aguviz: Unified pure vision agents for autonomous gui interaction. *arXiv preprint arXiv:2412.04454*, 2024.
- Yiheng Xu, Dunjie Lu, Zhennan Shen, Junli Wang, Zekun Wang, Yuchen Mao, Caiming Xiong, and Tao Yu. Agenttrek: Agent trajectory synthesis via guiding replay with web tutorials. In *International Conference on Learning Representations (ICLR)*, 2025.
- Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. An illusion of progress? assessing the current state of web agents. In *Conference on Language Modeling (COLM)*, 2025. arXiv:2504.01382.
- Rui Yang, Qianhui Wu, Zhaoyang Wang, Hanyang Chen, Ke Yang, Hao Cheng, Huaxiu Yao, Baoling Peng, Huan Zhang, Jianfeng Gao, et al. Gui-libra: Training native gui agents to reason and act with action-aware supervision and partially verifiable rl. *arXiv preprint arXiv:2602.22190*, 2026.
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. GPT-4V(ision) is a generalist web agent, if grounded. In *International Conference on Machine Learning (ICML)*, 2024a.
- Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyang Luo, Zhangchi Feng, and Yongqiang Ma. Llamafactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Bangkok, Thailand, 2024b.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.

A APPENDIX

Additional implementation details, dataset examples, and qualitative trajectories will be provided here.

A.1 TRAINING AND INFERENCE SYSTEM PROMPT

We fine-tune and evaluate the model with a single fixed system prompt that, at each step, conditions on the current page and emits the next action from the unified action space of Table 1. Using the same prompt for both *training* the model on the BrowserForge corpus and *evaluating* the fine-tuned model at inference time keeps the input format the model is trained on identical to the one it is tested on. The prompt fixes the coordinate convention (normalized integers in $[0, 1000]$), enumerates the available actions and their arguments, and specifies the exact three-line output format (Thought / Description / Action). At run time the trailing `{instruction}` and `{history}` placeholders are filled with the current task and the sequence of previous actions, and the current screenshot is supplied as the image input. The full prompt is shown below.

You are a browser GUI agent. Given an instruction, the current screenshot, and the history of previous actions, predict the SINGLE next action that best progresses toward completing the user's instruction.

Coordinates are normalized to the [0, 1000] range. Use integers in this range regardless of the actual image resolution.

Action Space

- click(coordinate): Click on a point on the screen. 'coordinate' is '[x, y]' with integers in [0, 1000].
- type(coordinate, text): Click on the input field at 'coordinate' [x, y] to focus it, then type 'text'.
- select(coordinate, value): Select a value from a dropdown/combobox. 'coordinate' is '[x, y]' of the dropdown element, 'value' is the option text to select.
- scroll(direction, amount): Scroll the visible page in the given direction by 'amount'. 'direction' is one of "up", "down", "left", "right" (screen-relative; "down" reveals content below the current view). 'amount' is a float in [0.05, 1.0] denoting the fraction of the viewport, e.g. 0.5 = half a screen.
- key(key): Press a single keyboard key, e.g. "Enter", "Tab", "Escape", "Backspace".
- wait(seconds): Wait for the page to load or settle. 'seconds' is a positive integer.
- go_back(): Navigate back to the previous page in the current tab.
- visit_url(url): Navigate the current tab to the given absolute URL.
- finish(message): Conclude the task and reply to the user. Use the prefix "[ANSWER] ..." to give the requested answer. Use exactly "[EXIT]" as the message if the answer was already provided in a previous step and you only need to terminate.

Output Format

Output exactly the following three lines, in this order, with no extra text before or after:

Thought: your reasoning about the current screenshot and the next step

Description: a short natural-language description of the action

Action: a single-line JSON object of the form {"action": "action_name", ...args}, where the fields match the action signature above.

Instruction
{instruction}

History
{history}